

Docker Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Institute: Archer Infotech, Kothrud, Pune - IT training since 2009

Website: <https://archerinfotech.in>

Contact: +91 9850 678451 | info@archerinfotech.in

Mode: Classroom at Kothrud, Pune and live-online batches

Duration: 1.5 Months

Level: Intermediate

Course page: <https://archerinfotech.in/courses/cloud-devops/docker-training-in-pune>

Course Overview

Docker is the foundational container technology every modern Pune software team ships with — from product engineering at Persistent Systems and BMC Software to BFSI workloads at Bajaj Finserv and Synechron, fintech / SaaS startups, and the captive engineering centres of BMW TechWorks, Mercedes-Benz R&D, Cummins, and John Deere ETC. Archer Infotech's Docker training in Pune teaches the platform as it is actually used in 2026 — Docker Engine and Docker Desktop, modern Dockerfile authoring with BuildKit and Buildx, multi-stage builds, distroless and Alpine-based minimal images, Docker Compose for local-dev stacks, plus container security with Trivy, image signing with cosign, and the production hygiene that prevents the most common 'works on my machine' failure modes. The course is the right standalone introduction for developers and the natural prerequisite for our Kubernetes and DevOps tracks. Classroom in Kothrud, online live, and weekend batches available.

Why Learn Docker in 2026

Docker is no longer a tool — it is the lingua franca of how software is shipped. Every Pune backend developer JD lists Docker as expected (not preferred); every DevOps / SRE / Platform Engineer JD lists it as a hard requirement; every Pune product company packages services in containers as the production default. Indeed Pune lists more than 1,500 active openings that mention Docker as a required skill, spanning Backend Developer, Full Stack Developer, DevOps Engineer, SRE, Cloud Engineer, and ML Engineer titles. The biggest employers are Persistent Systems, BMC Software, Bajaj Finserv, Synechron, BMW TechWorks India, Mercedes-Benz R&D India, Mastercard Pune Tech Hub, Cummins, John Deere ETC, plus the Pune SaaS / fintech scene (Amagi, Fyllo, BharatPe Pune, Razorpay Pune).

What changed in 2026: BuildKit is the production default for image builds (faster, parallel, secret-aware, cache-efficient), Docker Buildx ships multi-platform image builds out of the box (linux/amd64 + linux/arm64 for Apple Silicon and Graviton), distroless and Alpine-based minimal images have eclipsed full Ubuntu / Debian for production use cases, Docker Compose v2 (YAML-spec stable, no more docker-compose vs docker compose confusion), and Docker Scout has matured for built-in image scanning. Trivy remains the open-source default for security scans, and cosign is the standard for signing images in supply-chain-aware production. Apple Silicon native support is fully stable.

What this means for hiring: 2026 Pune Docker JDs expect Dockerfile authoring at the multi-stage level (not just FROM ubuntu and apt install), Docker Compose for local-dev fluency, image security awareness (non-root user, minimal base, vulnerability scanning), basic registry usage (Docker Hub, ECR, ACR, GHCR), and the ability to debug 'why doesn't my container start' from first principles. Senior roles add multi-platform builds, supply-chain hygiene (cosign, SBOM), and containerised CI/CD patterns. Archer Infotech's curriculum is rebuilt around exactly these expectations — production-realistic, security-aware, multi-platform-ready.

- 1,500+ Pune Indeed JDs list Docker as a required skill (May 2026)
- BuildKit + Buildx multi-platform — the 2026 production default
- Distroless / Alpine minimal images — security and image-size discipline

- Trivy + cosign + SBOM — supply-chain security baseline
- Right standalone course OR prerequisite for our Kubernetes / DevOps tracks

Who This Course Is For

This course fits you if

- Engineering, BCS, MCA, or BSc-CS student who wants to add Docker before joining a backend or DevOps role
- Working backend / full-stack developer wanting to add containerisation to your skill stack
- Working developer at a Pune company that is migrating to containers — you need to lead packaging credibly
- DevOps / sysadmin practitioner wanting Docker depth before tackling Kubernetes
- Working professional preparing to take our Kubernetes or DevOps course — Docker is the prerequisite
- QA engineer wanting to containerise test environments and CI pipelines

This course is not the right fit if

- If you have no Linux command-line comfort — basic Linux fluency is required from day 1; we run docker / bash / Linux constantly
- If you have no programming background — at least basic familiarity with one language is required
- If you cannot put in 6–8 hours per week of practice outside class — Docker is learned by building and breaking, not watching
- If you only want a certificate sticker with no portfolio — Pune hiring screens hard on real Dockerfiles and deployed images
- If you have 2+ years of production Docker / multi-stage build authoring with BuildKit — you'll be under-stretched; jump directly to our Kubernetes or DevOps course

Detailed Curriculum

Module 1: Containers, Images & the Docker Engine (Week 1)

What a container actually is, established properly before any command is typed. Containers versus virtual machines as an isolation and resource-sharing story, the Linux namespaces and cgroups underneath (intuition rather than kernel internals), and the Docker Engine architecture — CLI, daemon, containerd and runc — so that later failures can be attributed to the right layer.

The OCI image format is opened up: layers, the manifest, the config, and content-addressable digests. The week ends with fluency in the core CLI and an understanding of why docker run on the same tag can produce different results tomorrow.

- Containers vs VMs — the practical distinction
- Linux namespaces and cgroups (intuition only)
- Docker Engine architecture — daemon, containerd, runc
- OCI image format — layers, manifest, config, digests
- Tags vs digests and reproducibility
- Docker Desktop vs Docker Engine — Linux, macOS, Windows
- Core CLI — pull, run, exec, logs, ps, inspect, stop, rm
- Container lifecycle and exit codes

Module 2: Dockerfile Authoring & the Build Cache (Week 2)

Writing Dockerfiles that build fast and behave predictably. Every instruction and when each is the right one, then the subject that matters most in daily work: the layer cache, how instruction ordering determines whether a one-line code change rebuilds in four seconds or four minutes, and .dockerignore as both a speed and a secret-safety control.

The distinctions that trip people up get dedicated time — ARG versus ENV, ENTRYPOINT versus CMD, shell versus exec form — along with signal handling, graceful shutdown and PID 1 behaviour.

- Every Dockerfile instruction and when each is right
- Layer cache mechanics and instruction ordering
- .dockerignore for speed and secret safety
- ARG vs ENV — build-time vs runtime
- ENTRYPOINT vs CMD, shell vs exec form

- PID 1, signal handling and graceful shutdown
- HEALTHCHECK and readiness semantics
- USER and the non-root discipline
- Labels and image metadata

Module 3: BuildKit, Multi-Stage Builds & Minimal Images (Week 3)

Modern image building. BuildKit as the default builder — parallel stage execution, cache mounts that persist dependency downloads between builds, and build secrets that never land in a layer (the correct answer to a question most candidates get wrong).

Multi-stage builds separate the toolchain from the runtime, then base-image selection is treated as an engineering decision: Alpine versus slim versus distroless, weighed on attack surface, glibc compatibility and debuggability. Language-specific patterns for Java, Node, Python, Go and .NET close the module.

- BuildKit — parallel stages and modern build features
- Cache mounts for dependency downloads
- Build secrets that never reach a layer
- Multi-stage builds — builder and runtime separation
- Alpine vs slim vs distroless — choosing a base
- glibc vs musl compatibility pitfalls
- Language patterns — Java, Node, Python, Go, .NET
- Measuring and reducing image size
- Reproducible builds — lock files and pinned digests

Module 4: Multi-Platform Builds & Registries (Week 3)

Shipping images that run everywhere and live somewhere. Buildx with QEMU emulation and native builders for amd64 and arm64 output — now a routine requirement given Apple Silicon laptops and Graviton or Ampere servers — plus manifest lists and how a single tag serves multiple architectures.

Registries follow: Docker Hub rate limits, Amazon ECR, Azure Container Registry, Google Artifact Registry and GHCR; authentication, tagging strategy, retention and lifecycle policies, and mirroring or pull-through caches for teams behind slow links.

- Buildx builders, QEMU emulation and native nodes
- Multi-platform images for amd64 and arm64
- Manifest lists and multi-arch tags
- Docker Hub rate limits and their consequences
- ECR, ACR, Artifact Registry and GHCR
- Registry authentication and credential helpers
- Tagging strategy — semantic, git-sha, immutable
- Retention, lifecycle policies and storage cost
- Pull-through caches and registry mirrors

Module 5: Docker Compose for Local Development (Week 4)

Compose v2 as the tool that makes a multi-service project runnable with one command. Services, networks and volumes; depends_on with health-gated conditions so a web container waits for a database that is genuinely ready rather than merely started; and environment files, variable interpolation and Compose secrets.

Profiles allow opt-in services, and override files handle per-environment differences without duplicating the base file.

Reference stacks — Node with Postgres, Java with Postgres and Kafka, Python with Celery and Redis — are built and debugged in class.

- Compose v2 — services, networks, volumes
- depends_on with health-conditioned waits
- Healthchecks and startup ordering
- Environment files and variable interpolation

- Compose secrets and configs
- Profiles for opt-in services
- Override files for environment-specific config
- Reference stacks — Node+PG, Java+PG+Kafka, Python+Celery
- watch mode and developer inner loop

Module 6: Networking — Drivers, DNS & Service Discovery (Week 5)

Container networking from the driver up. Bridge, host, overlay, macvlan and none, and the workload each one suits; user-defined bridge networks and why they behave differently from the default bridge; and embedded DNS resolution that lets containers address each other by service name.

Port publishing and the host-to-container mapping are covered alongside the diagnostics that resolve most connectivity problems — inspecting networks, testing from inside a container, and distinguishing a DNS failure from a firewall one.

- Network drivers — bridge, host, overlay, macvlan, none
- User-defined bridges vs the default bridge
- Embedded DNS and container name resolution
- Port publishing and host mapping
- Multi-network containers and network aliases
- Overlay networking and Swarm mode overview
- Diagnosing connectivity from inside a container
- Telling DNS failures apart from firewall failures

Module 7: Volumes, Storage & Stateful Containers (Week 5)

Persisting data correctly. Named volumes versus bind mounts versus tmpfs, ownership and permission problems on Linux (the single most common source of 'it works on my Mac'), and volume drivers for remote or networked storage.

Stateful patterns are then built for real: PostgreSQL, Redis and Elasticsearch containers with durable storage, initialisation scripts, and backup and restore performed through a helper container. The module ends on data migration between volumes and honest guidance on running databases in containers in production.

- Named volumes, bind mounts and tmpfs
- Ownership, UID/GID and permission pitfalls
- Volume drivers and remote storage
- Stateful patterns — Postgres, Redis, Elasticsearch
- Initialisation scripts and seed data
- Backup and restore through helper containers
- Migrating data between volumes
- When not to run a database in a container

Module 8: Container Security & Hardening (Week 6)

Reducing what a compromised container can do. Non-root users and read-only root filesystems, dropping Linux capabilities to the minimum set, seccomp and AppArmor profiles, and no-new-privileges. Resource limits as a denial-of-service control rather than only a performance setting.

The threat model is made explicit — what container isolation does and does not protect against, why the Docker socket is effectively root, and how privileged containers and host mounts undo everything else. Rootless Docker and gVisor close the module.

- Non-root users and read-only root filesystems
- Capability dropping and the minimum viable set
- seccomp and AppArmor profiles
- no-new-privileges and privilege escalation
- Resource limits as a DoS control
- The Docker socket as a root-equivalent risk
- Privileged containers and host mounts
- Rootless Docker and gVisor

- What container isolation does not protect against

Module 9: Supply Chain — Scanning, SBOMs & Signing (Week 6)

Proving an image is trustworthy, which is now a routine requirement in Indian enterprise and BFSI delivery. Vulnerability scanning with Trivy and Docker Scout, reading a CVE report critically, and distinguishing findings that matter from base-image noise you cannot fix.

SBOM generation with Syft or Docker Scout, provenance and SLSA build attestations, and image signing with Cosign — including keyless signing — followed by verification enforced at deploy time so an unsigned image simply cannot run.

- Vulnerability scanning with Trivy and Docker Scout
- Reading CVE reports and triaging findings
- Base-image noise vs actionable vulnerabilities
- SBOM generation with Syft and Docker Scout
- Provenance and SLSA build attestations
- Image signing with Cosign, including keyless
- Verification and admission-time enforcement
- Pinning digests and dependency hygiene

Module 10: CI/CD, Image Publishing & Layer Caching (Week 6)

Automating everything built so far. GitHub Actions building and publishing images with OIDC federation to a cloud registry — no static credentials — plus GitLab CI as the alternative many Pune enterprise teams run.

Build speed is treated as a first-class concern: registry-backed layer caching, GitHub Actions cache backends, and matrix builds across architectures. The pipeline is completed with scan and signature gates that fail the build, then tag promotion from development through to release.

- GitHub Actions with OIDC for keyless registry publish
- GitLab CI equivalents and runners
- docker/build-push-action and buildx in CI
- Registry-backed and GHA layer caching
- Matrix builds across architectures
- Scan and signature gates that fail the build
- Tag promotion — dev, staging, release
- Build-time secrets handled safely in CI

Module 11: Production Operations & Troubleshooting (Week 7)

Running containers once they leave your laptop. Logging drivers and log rotation, container metrics and docker stats, restart policies, and the health-check-driven restart behaviour orchestrators depend on.

Troubleshooting is taught as a repeatable method rather than a list of tricks: diagnosing OOM kills from exit code 137, debugging a container that exits immediately, inspecting a distroless image with no shell using ephemeral debug containers, and reading layer history to find what actually changed. Compose-to-Kubernetes migration closes the module.

- Logging drivers, log rotation and log shipping
- Container metrics, docker stats and cAdvisor
- Restart policies and health-check-driven restarts
- Diagnosing OOM kills and exit code 137
- Debugging containers that exit immediately
- Debugging distroless images without a shell
- Reading image history and layer diffs
- Resource tuning — CPU shares, memory limits
- From Compose to Kubernetes — what carries over

Module 12: Capstone Project & Interview Preparation (Week 7 + placement prep)

The capstone brings the whole course together: a multi-service application containerised with multi-stage, multi-platform builds, orchestrated locally with Compose, hardened to a non-root read-only baseline, scanned and signed in a CI pipeline,

and published to a registry with a documented tagging strategy.

Interview preparation targets what Pune panels actually ask about Docker — image-size reduction, cache-miss diagnosis, the ENTRYPOINT and CMD distinction, security hardening, and the Docker socket question. Resume, LinkedIn and GitHub polish included.

- Capstone implementation, deployment and README
- Documented tagging and release strategy
- Image-size reduction exercise with measured results
- Cache-miss diagnosis under interview conditions
- Security hardening walkthrough
- Common Docker interview questions and model answers
- Resume and LinkedIn rewrite
- GitHub portfolio polish
- HR mock interview and salary negotiation

Module 13: AI-Assisted Development Workflow (Final Week)

The skill every 2026 hiring panel now probes for — building real work with AI in the loop, responsibly. Learn to drive AI assistants (GitHub Copilot, Claude, Cursor, and IDE-native AI) to scaffold and accelerate the tools and stack this course covers, generate tests, explain and refactor unfamiliar code, and cut the boilerplate — while keeping you firmly in control of every decision. Heavy focus on guardrails: reviewing each AI suggestion, spotting hallucinated APIs or wrong answers, and handling licensing and data-privacy concerns. Close with a mini-project that takes a deliverable end-to-end using an AI-assisted workflow, then fold the same tooling into version control and everyday team practice.

- AI assistants — GitHub Copilot, Claude, Cursor, IDE-native AI
- Effective prompting for this course's stack — scaffolding, boilerplate, config
- AI-assisted test generation and coverage
- Explaining, refactoring, and modernising unfamiliar code with AI
- AI-driven review, error detection, and quality checks
- Generating and maintaining documentation with AI
- AI debugging — interpreting errors, logs, and failing output
- Guardrails — reviewing output, avoiding hallucinations, licensing & data privacy
- Team workflow — AI in the editor, in reviews, and in delivery pipelines
- Mini-project — a deliverable built end-to-end with an AI-assisted workflow

Capstone Projects

Production-Grade Containerisation of a Real Application

Take a real backend application (Spring Boot, FastAPI, Node.js / Express, or .NET — your choice or one we provide) and containerise it from scratch the production-correct way — multi-stage Dockerfile (builder + runtime), distroless or Alpine runtime image, non-root user, healthcheck, signal handling for graceful shutdown, multi-platform build (linux/amd64 + linux/arm64 via Buildx), Trivy scan in the CI pipeline, cosign-signed image pushed to GHCR via GitHub Actions OIDC. Plus a Docker Compose stack pairing the app with Postgres + Redis for local dev. Outcome: a public GitHub repository with the Dockerfile, Compose file, GitHub Actions pipeline, and a clickable image registry link — exactly what Pune backend / DevOps hiring panels look at first.

- Technology: Docker Engine + BuildKit
- Technology: Docker Buildx for multi-platform
- Technology: Distroless or Alpine runtime images
- Technology: Docker Compose v2
- Technology: Trivy + Docker Scout
- Technology: cosign for signing
- Technology: GitHub Actions with OIDC
- Technology: GitHub Container Registry (GHCR)

Local-Dev Compose Stack for a Microservices Application

A complete docker compose stack for a small microservices system — three services (web frontend, API backend, worker), Postgres + Redis + RabbitMQ, healthchecks driving service-startup ordering via depends_on conditions, Compose profiles for optional services (Adminer for DB inspection, MailHog for email testing), .env-driven configuration, plus a small bootstrap script that seeds the database. The stack must come up in under 60 seconds with docker compose up. Demonstrates the patterns Pune product engineering teams hire on for local-dev tooling.

- Technology: Docker Compose v2
- Technology: Multi-service stack with healthchecks
- Technology: Postgres + Redis + RabbitMQ
- Technology: Compose profiles
- Technology: Adminer + MailHog for dev tooling
- Technology: Bootstrap scripts

Multi-Architecture Image Pipeline for OSS Project

Pick or fork a real open-source project and add a complete container image pipeline — multi-stage Dockerfile, multi-platform builds via Buildx (linux/amd64 + linux/arm64 + linux/arm/v7 for IoT), GitHub Actions matrix-build pipeline that publishes signed images to GHCR on every release tag, automated Trivy + cosign workflow, and an SBOM attached to each image. Includes a small README that shows users how to pull and verify the signature. Optional: contribute the pipeline back to the upstream project. Demonstrates the patterns Pune product engineering teams expect on senior containerisation work.

- Technology: Docker Buildx multi-platform
- Technology: GitHub Actions matrix builds
- Technology: GHCR with signed images
- Technology: Trivy + cosign in CI
- Technology: SBOM generation
- Technology: Conventional Commits + semantic-release

Career Outcomes

Docker fluency is now expected on every modern Pune developer JD, not optional. Indeed Pune lists 1,500+ active openings that list Docker as required, spanning backend, full-stack, DevOps, SRE, Cloud Engineer, and ML Engineer titles. Docker alone rarely makes you a 'Docker engineer' — but it is the foundation that opens all of those job families. The biggest employers across that range are Persistent Systems, BMC Software, Bajaj Finserv, Synechron, BMW TechWorks India, Mercedes-Benz R&D India, Mastercard Pune Tech Hub, Cummins, John Deere ETC, plus the Pune SaaS / fintech scene.

What pulls a candidate above the median band on Docker fluency: a public GitHub repository with a real multi-stage Dockerfile + multi-platform build pipeline, demonstrable security awareness (non-root, minimal base, Trivy scan), and one production deployment story. Our capstone projects are designed exactly around these signals.

Docker is the prerequisite for higher-paying specialisations — Kubernetes Engineer, Platform Engineer, SRE — where compensation runs significantly above generic Backend / Cloud titles. Most of our Docker graduates take our Kubernetes or DevOps course within 3–6 months as the depth specialisation.

Roles after this course

- Backend Developer (with Docker fluency)
- Junior DevOps Engineer
- Junior Cloud Engineer
- Build / Release Engineer
- QA Automation Engineer (with containerised tests)
- Software Engineer with platform-engineering trajectory
- Prerequisite met for Kubernetes / DevOps / SRE specialisations

Indicative salary bands

- Backend Developer (Pune, Docker required) - Rs. 6,93,500 per year average (source: Indeed Pune (Backend Developer))
- DevOps Engineer (Pune) - Rs. 7,07,950 per year average (source: Indeed Pune (DevOps Engineer))
- Cloud Engineer (Pune) - Rs. 6,72,490 per year average (source: Indeed Pune (Cloud Engineer))
- Kubernetes Administrator (Pune, Docker prerequisite) - Rs. 8,40,000 per year average (source: Indeed Pune (Kubernetes Administrator))
- Senior SRE / Platform Engineer (national, 5–8 years) - Rs. 18,00,000 – Rs. 30,00,000 per year (source: 6figr India Senior SRE (Pune ±10%))

Companies hiring for these skills

- Persistent Systems
- BMC Software
- Bajaj Finserv
- Synechron
- BMW TechWorks India
- Mercedes-Benz R&D India
- Mastercard Pune Tech Hub
- Cummins India
- John Deere ETC
- Honeywell
- Amagi
- Fylo
- TCS
- Infosys
- Cognizant
- Capgemini

Batch Modes and Duration

Duration: 6 weeks of structured curriculum plus 1 week of capstone project and interview preparation (~1.5 months total)

Classroom

- Location: Archer Infotech, Kothrud, Pune
- Morning batch — 10:00 to 13:00
- Evening batch — 18:00 to 21:00
- Lab access available outside class hours

Live online

- Same hours as classroom batches — morning or evening
- Recordings available for review
- Same lab reviews and project feedback as in-person batches
- Zoom for live sessions
- Docker Desktop or Docker Engine per student
- GitHub for Dockerfile and Compose reviews
- Slack / WhatsApp for asynchronous Q&A

Weekend

- Saturday + Sunday, 09:00 to 13:00
- Stretches over ~2.5 months instead of 1.5 to accommodate working professionals. Same content, lower weekly load.

Maximum 15 students per batch — small enough that the trainer reviews every student's Dockerfiles and Compose stacks personally. Classroom batches start every 3 weeks; weekend batches every 5 weeks.

Fees

Course fees range from Rs. 20,000 to Rs. 90,000 depending on mode (classroom / online / weekend), batch type, and any applicable concession — Docker as a 1.5-month foundational course typically lands at the lower-mid end of that range. Kindly reach us for the current 2026 quote — we calibrate by early-bird timing, group enrolment, and returning-alumni concessions. Most students take Docker as a stepping stone to Kubernetes / DevOps; ask about the bundled Docker + Kubernetes pricing for combined enrolment.

Indicative range: Rs. 20,000 – Rs. 90,000 (Docker, as a focused 1.5-month course, typically lands at the lower end of the range; combined Docker + Kubernetes / DevOps enrolment offers significant bundle savings)

- Single payment with early-bird discount
- EMI in 2 instalments at no extra cost
- Corporate sponsorship — invoiced to your employer with GST
- Bundled Docker + Kubernetes or Docker + DevOps enrolment with discount

Placement Support

Docker alone is a foundational skill rather than a standalone hireable specialisation; placement focus for Docker graduates is calibrated accordingly. By the end of the curriculum your resume highlights real multi-stage Dockerfile authoring, your GitHub has at least one production-style containerisation repository, and you have completed at least two mock technical interviews focused on backend / DevOps roles where Docker is part of the broader stack.

We say placement support, not placement guarantee — for two honest reasons. First, no institute can guarantee a hire when the final decision is the company's. Second, Docker alone rarely lands the offer; it is the foundation that strengthens applications for backend, DevOps, or platform roles. Our support is unconditional, time-bound (six months after course completion), and includes free re-entry to a future batch's interview-prep sessions. Most Docker graduates progress directly to our Kubernetes or DevOps course for the depth specialisation that turns Docker fluency into a hireable role.

The placement process

- Week 5 — resume and LinkedIn rewrite, highlighting Docker fluency for backend / DevOps JDs
- Week 6 — GitHub portfolio cleanup, Dockerfile / Compose / image registry links
- Week 7 — two rounds of mock technical interviews (Docker-aware backend / DevOps angles)
- Week 7 — HR mock interview and salary negotiation coaching
- Post-course — referrals via our 17-year alumni network at 12+ partner companies
- Strong recommendation to enrol in our Kubernetes or DevOps course as the natural next step
- Up to 6 months of continued support after course end
- Free re-entry to future batch interview-prep sessions if first round does not land

Partner companies

- Persistent Systems
- BMC Software
- Bajaj Finserv
- Synechron
- BMW TechWorks India
- Mercedes-Benz R&D India
- Mastercard Pune Tech Hub
- Cummins
- TCS
- Infosys
- Cognizant
- Capgemini

Prerequisites and How to Start

Prerequisites: basic Linux command-line comfort, basic familiarity with at least one programming language (any — C, C++, Java, Python, JavaScript, Go, .NET), and willingness to commit 6–8 hours per week of practice outside class. We do NOT

require prior container experience — we start from `docker pull hello-world` in week 1. If you are a working backend or full-stack developer, you are well-prepared; pure non-developers should do a Linux + scripting foundation first.

Suggested first steps

- Decide your mode — classroom in Kothrud, online live, or weekend
- Check the upcoming batch dates on our batch schedule page
- Book a free 30-minute counselling call — we will honestly tell you whether the course fits your goal (we say no to roughly 10% of enquirers because Linux foundation isn't yet there)
- Confirm enrolment and complete pre-course orientation (Docker Desktop or Docker Engine install scripts, GitHub account, VS Code setup)
- Show up to day one with a laptop running 64-bit Linux / macOS / Windows-with-WSL2, 8GB+ RAM, and Docker Desktop or Docker Engine pre-installed (we provide an install script)

Frequently Asked Questions

Which is the best Docker training institute in Pune?

We can't honestly answer 'best' for ourselves. The test that works: ask any institute you are considering to (1) name the trainer who will teach your batch and show their LinkedIn, (2) show real student Dockerfiles and signed images published to a public registry, and (3) name companies that hired their last 5 batches (or where their Docker graduates went on to take Kubernetes / DevOps). Compare on those three.

How long does Docker training in Pune take at Archer Infotech?

Approximately 1.5 months — 6 weeks of structured curriculum plus 1 week of capstone project and interview preparation. The weekend batch stretches over ~2.5 months at the same content depth, designed for working professionals.

What is the salary impact of learning Docker?

Docker fluency is foundational rather than a standalone specialisation — but it strengthens applications for Backend Developer (Pune average Rs. 6.94 lakh on Indeed), DevOps Engineer (Rs. 7.08 lakh), Cloud Engineer (Rs. 6.72 lakh), and as the prerequisite for Kubernetes Administrator (Rs. 8.40 lakh). Senior SRE / Platform Engineer roles where Docker / Kubernetes are core run Rs. 18–30 lakh nationally.

What is the fee for the Docker course in Pune?

Course fees range from Rs. 20,000 to Rs. 90,000 depending on mode (classroom / online / weekend), batch type, and applicable concession. Docker as a 1.5-month focused course typically lands at the lower end of this range. We offer significant bundle discounts when Docker is enrolled alongside our Kubernetes or DevOps courses — talk to us about combined pricing.

Do I need Linux experience to learn Docker?

Yes — basic Linux command-line comfort is required from day 1. We do not start from 'what is a terminal'. If you are new to Linux, work through a basic Linux course first; we politely turn away candidates who are not yet at the basic-Linux level.

Should I take Docker before Kubernetes?

Yes — Docker is the prerequisite. Our Kubernetes course assumes Docker fluency from day 1; skipping Docker and going straight to Kubernetes wastes the first three weeks because every Kubernetes concept builds on container fundamentals. Many of our DevOps students take Docker -> Kubernetes back-to-back; the combined learning is ~4 months and produces a hire-ready DevOps / Platform Engineer profile.

Will I work on real projects?

Yes — three capstone projects: (1) production-grade containerisation of a real backend application with multi-stage build, multi-platform image, signed publish to GHCR, (2) local-dev Compose stack for a microservices system, (3) multi-architecture image pipeline for an OSS project. All three become public GitHub repositories with images published to a public registry that you can reference in interviews.

Are BuildKit and Buildx covered?

Yes — BuildKit (the modern build engine) is treated as the production default throughout the course, not as 'advanced'. Buildx for multi-platform builds (linux/amd64 + linux/arm64 + linux/arm/v7) is covered hands-on, with Apple Silicon students cross-building amd64 images and amd64 students cross-building arm64. Multi-platform builds are increasingly expected on Pune product engineering JDs.

Is image security / supply-chain coverage included?

Yes — week 6 covers Trivy and Docker Scout for vulnerability scanning, cosign for image signing, SBOM generation, plus the run-time defences (non-root user, read-only root filesystem, capability dropping, seccomp). Image security has become a standard ask on Pune DevSecOps and senior backend interviews; we cover it deeply enough that you can defend the choices.

Is Docker Compose covered?

Yes — week 4 is a full module on Docker Compose v2 (the modern docker compose plugin, no more standalone docker-compose v1). Capstone Project #2 is a complete multi-service Compose stack. Compose remains the right tool for local-dev and small-scale production deployments; we cover it deeply enough that you can stand up reference stacks for any common backend pattern.

Are weekend Docker classes available in Pune?

Yes — Saturday and Sunday, 09:00–13:00, stretched over ~2.5 months instead of 1.5. Same content, same trainer, same labs and capstone. Designed for working professionals who cannot attend weekday batches.

Should I take this course or your DevOps course?

Take this Docker course if you want a focused 1.5-month foundational on containers specifically. Take our DevOps course (3 months) if you want the broader programme — Linux + Git + Docker + Kubernetes + CI/CD + Terraform + Observability + Security + FinOps. Most candidates targeting DevOps / Platform / SRE roles take the DevOps course; Docker as a standalone fits working backend developers who need containerisation depth without the broader DevOps scope.

What about Docker Swarm?

Docker Swarm is briefly covered as a comparison point with Kubernetes but is not the focus. Pune production deployments have overwhelmingly standardised on Kubernetes (Swarm has effectively been retired by Mirantis, the current Docker maintainer). We cover Swarm at the level of 'know what it is, why teams have moved off it' — not as a production tool you should adopt in 2026.

What support do I get after course completion?

Six months of active placement support — mock interviews calibrated for backend / DevOps roles where Docker fluency is part of the stack, referrals via our alumni network at 12+ partner companies, resume / LinkedIn / GitHub rewrites, and salary negotiation coaching. Most importantly, strong recommendation and bundled discount to enrol in our Kubernetes or DevOps course as the natural depth specialisation.

Is the named trainer actually teaching, or are they just on the brochure?

Amol Patil personally leads every session of every batch from Day 1 through capstone — he writes Dockerfiles and Compose stacks daily and brings real production patterns into the classroom. The same name on this page is the same person you meet on day one; his LinkedIn is on the trainer profile page, and we welcome a 30-minute conversation with him before you enrol.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.